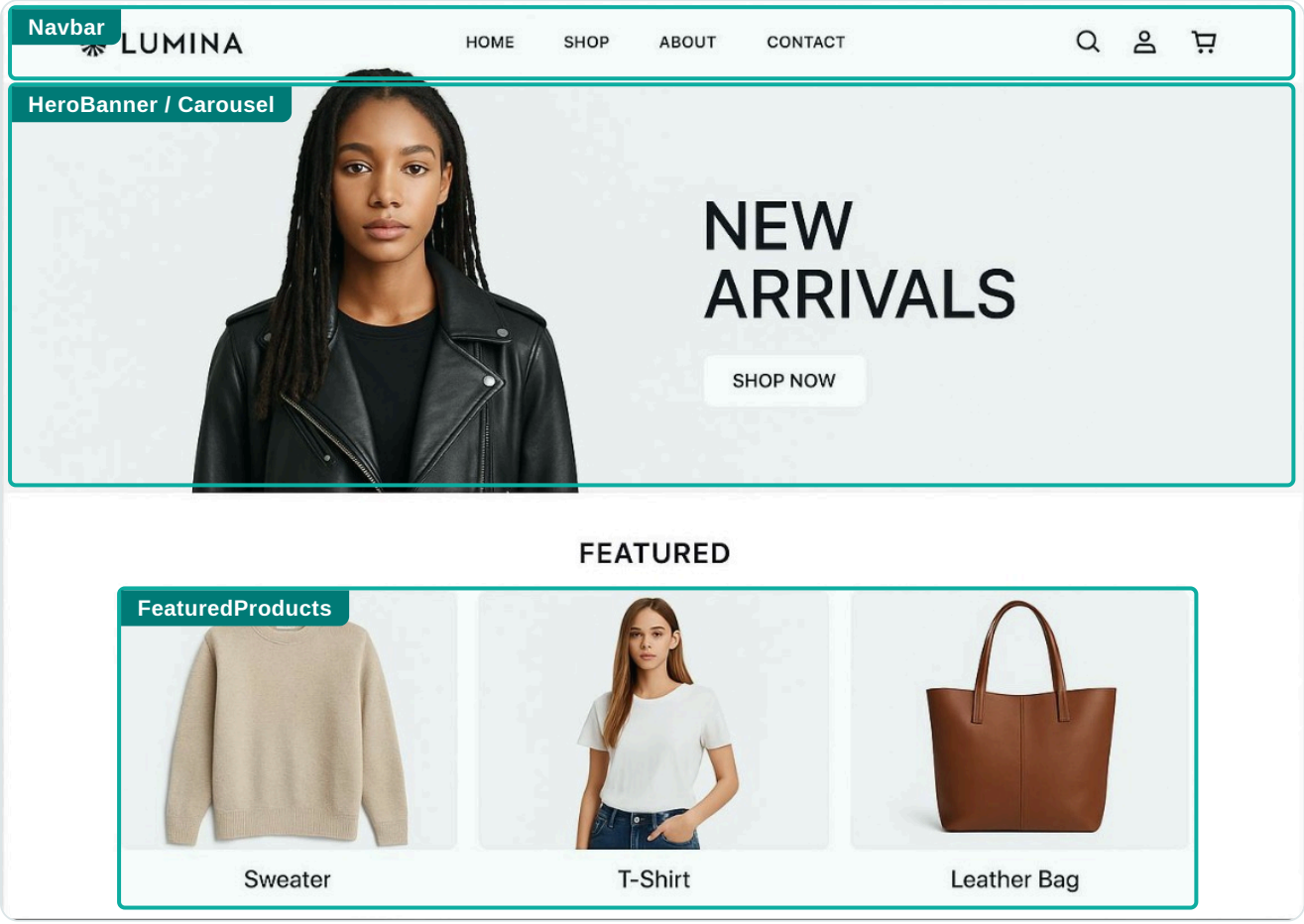
LEARNING GOAL

One screen. Many React components.

The LUMINA store is a useful example: what looks like one web page is actually made of small, independent UI pieces.



01 / Navbar

A reusable menu shown on multiple pages.

02 / HeroBanner

The main welcome section. It can become a carousel later.

03 / ProductCard

A repeated card for each product inside FeaturedProducts.

React component = a JavaScript function that returns UI. It has one clear responsibility, can receive data through **props**, and can be reused in many places.

FIRST BUILDING BLOCKS

Start with two simple components.

Each component has a single job. A filename like **Navbar.jsx** makes the responsibility easy to find.

1. Navbar.jsx

Navigation

Use **Link** to navigate without reloading the entire React app.

```
import { Link } from "react-router-dom";

export default function Navbar() {
  return (
    <nav>
      <strong>JUNIOR STORE</strong>
      <Link to="/">Home</Link>
      <Link to="/shop">Shop</Link>
    </nav>
  );
}
```

> **Notice:** `Navbar()` is a regular JavaScript function. Its return value is **JSX**, which looks similar to HTML.

2. ProductCard.jsx

Reusable UI

The same card can display different products by receiving **props**.

```
export default function ProductCard({ name, price }) {
  return (
    <article className="product-card">
      <div className="image-placeholder">Image</div>
      <h3>{name}</h3>
      <p>${price}</p>
    </article>
  );
}
```

What are props?

Inputs passed from a parent component into a child. Here, **name** and **price** change the content.

Why reuse?

One ProductCard definition produces many matching cards. Fix it once; all cards improve.

PUT THE PIECES TOGETHER

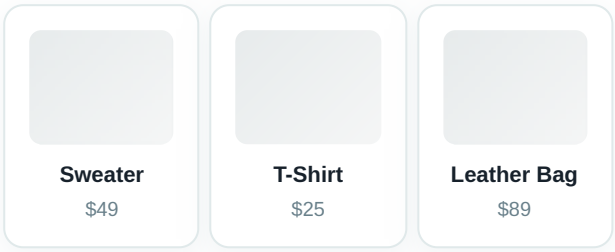
Build the Home page.

A parent component renders smaller child components. This is called **component composition**.

SIMPLE COMPONENT TREE

```
App
├── Navbar
├── Routes
│   └── Home
│       ├── HeroBanner
│       └── FeaturedProducts
│           └── ProductCard × 3
└── Footer
```

THREE CARDS / ONE COMPONENT



For Sprint 1, color boxes are enough. No product photos or API requests are needed.

FeaturedProducts.jsx

map() + props

```jsx
import ProductCard from "./ProductCard";

const products = [
  { id: 1, name: "Sweater", price: 49 },
  { id: 2, name: "T-Shirt", price: 25 },
  { id: 3, name: "Leather Bag", price: 89 },
];

export default function FeaturedProducts() {
  return (
    <section>
      <h2>Featured</h2>
      <div className="product-grid">
        {products.map((product) => (
          <ProductCard key={product.id}
            name={product.name} price={product.price} />
        ))}
      </div>
    </section>
  );
}
```

Home.jsx

Composition

```jsx
import HeroBanner from "../components/HeroBanner";
import FeaturedProducts from "../components/FeaturedProducts";

export default function Home() {
  return <><HeroBanner /><FeaturedProducts /></>;
}
```

> **What to understand:** **map()** converts an array into cards; **props** provide each card's data; **key** identifies each item in the list. Navbar and Footer can live in **App** because multiple pages share them.

Review challenge: explain how data moves from the products array into ProductCard.